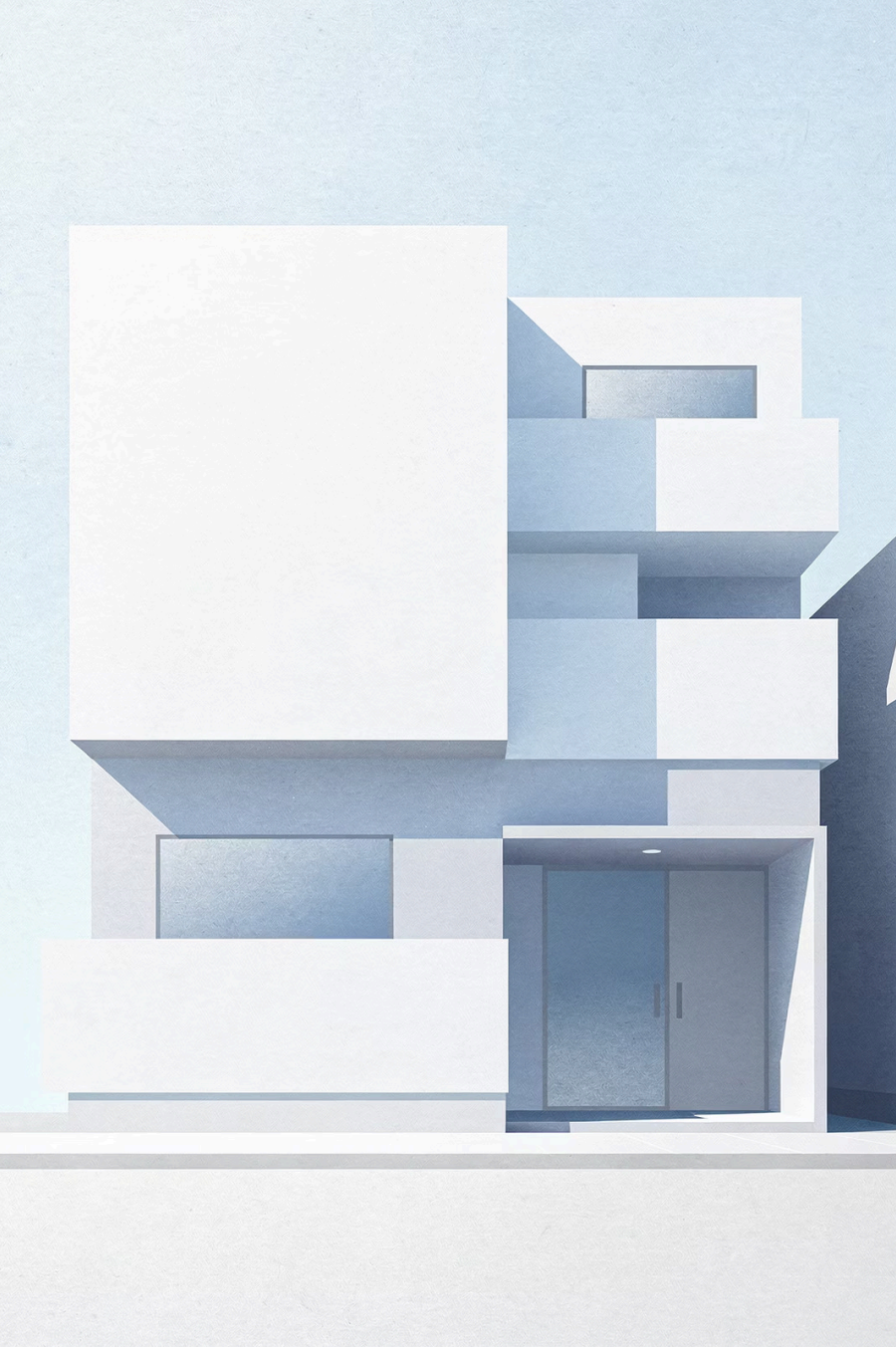




PATRÓN DE DISEÑO ESTRUCTURAL

Facade

También llamado: **Fachada**

Una interfaz simplificada para sistemas complejos. Descubre cómo este patrón transforma la forma en que interactúas con bibliotecas, frameworks y subsistemas.

Agenda de la Presentación

01

Propósito y Problema

Qué resuelve el patrón Facade y por qué es necesario

02

Solución y Analogía

Cómo funciona la fachada y su equivalente en el mundo real

03

Estructura y Pseudocódigo

Diagrama de clases y ejemplo completo de implementación

04

Aplicabilidad e Implementación

Cuándo usar el patrón y pasos para implementarlo

05

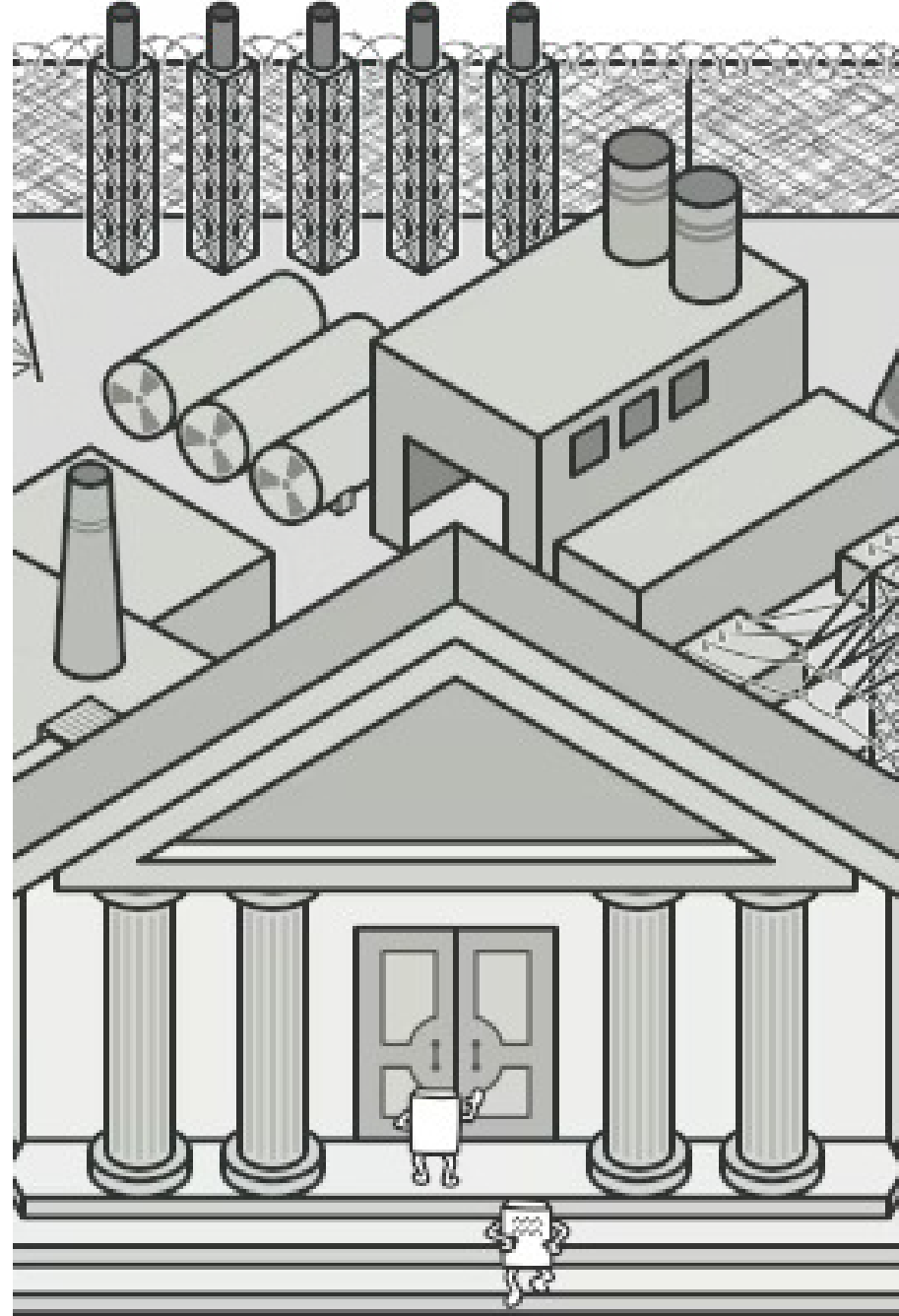
Pros, Contras y Relaciones

Ventajas, desventajas y vínculos con otros patrones

Propósito del Patrón Facade

Facade es un patrón de diseño estructural que proporciona una **interfaz simplificada** a una biblioteca, un framework o cualquier otro grupo complejo de clases.

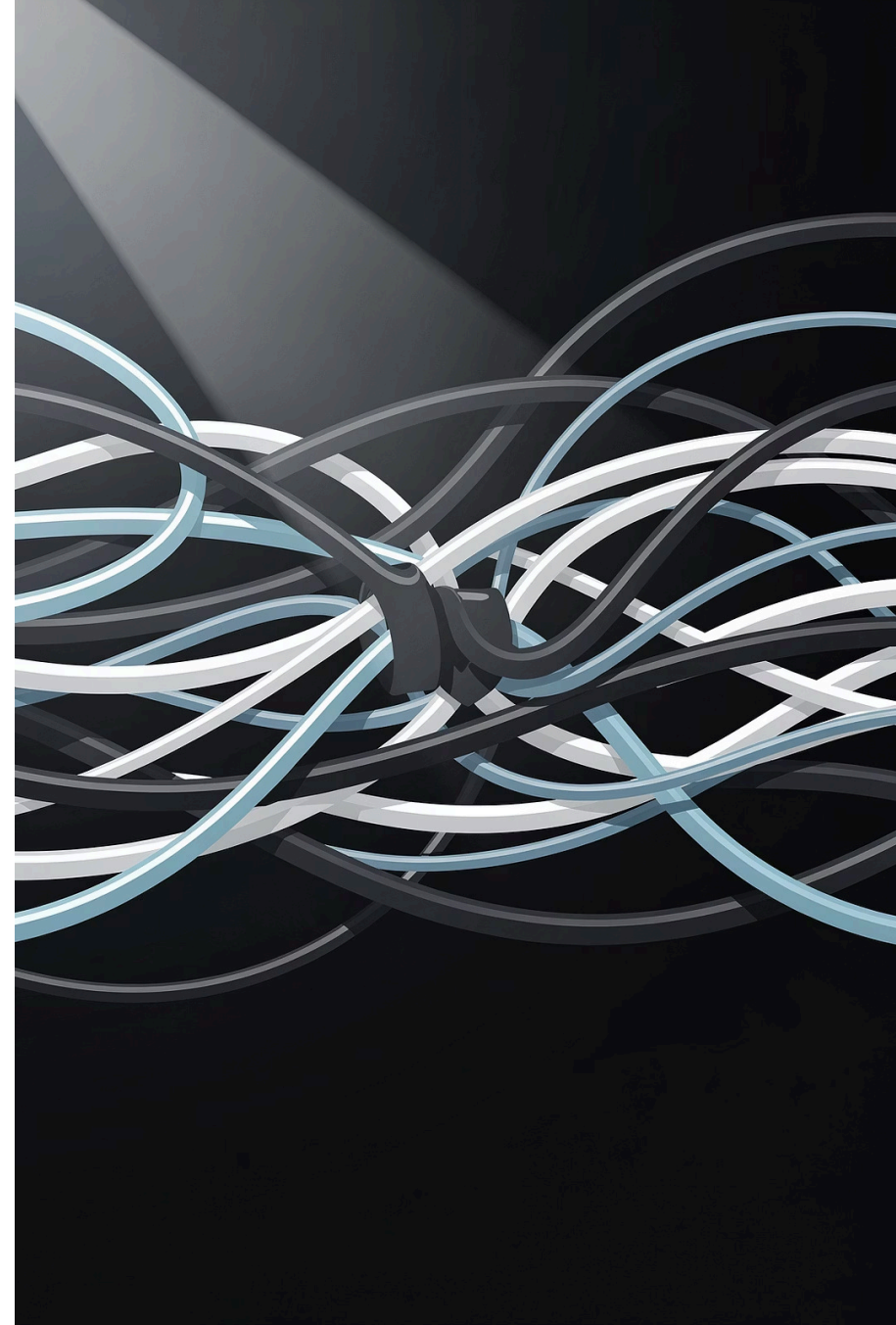
Su objetivo principal es reducir la complejidad visible para el código cliente, ofreciendo un punto de acceso único y sencillo a funcionalidades que internamente involucran múltiples objetos y dependencias.



El Problema

Imagina que debes lograr que tu código trabaje con un **amplio grupo de objetos** que pertenecen a una sofisticada biblioteca o framework. Normalmente, debes:

- Inicializar todos esos objetos
- Llevar un registro de las dependencias
- Ejecutar los métodos en el orden correcto



Consecuencia del Problema

Como resultado, la **lógica de negocio** de tus clases se vería estrechamente acoplada a los detalles de implementación de las clases de terceros, haciéndola:

Difícil de comprender

El código mezcla lógica propia con detalles del framework

Difícil de mantener

Cualquier cambio en el subsistema impacta todo el código



La Solución: El Patrón Facade

Una fachada es una clase que proporciona una **interfaz simple** a un subsistema complejo que contiene muchas partes móviles. Puede proporcionar una funcionalidad limitada en comparación con trabajar directamente con el subsistema, pero tan solo incluye las funciones **realmente importantes** para los clientes.

- ❏ Tener una fachada resulta útil cuando tienes que integrar tu aplicación con una biblioteca sofisticada con decenas de funciones, de la cual sólo necesitas una pequeña parte.

Ejemplo Práctico de la Solución

El Escenario

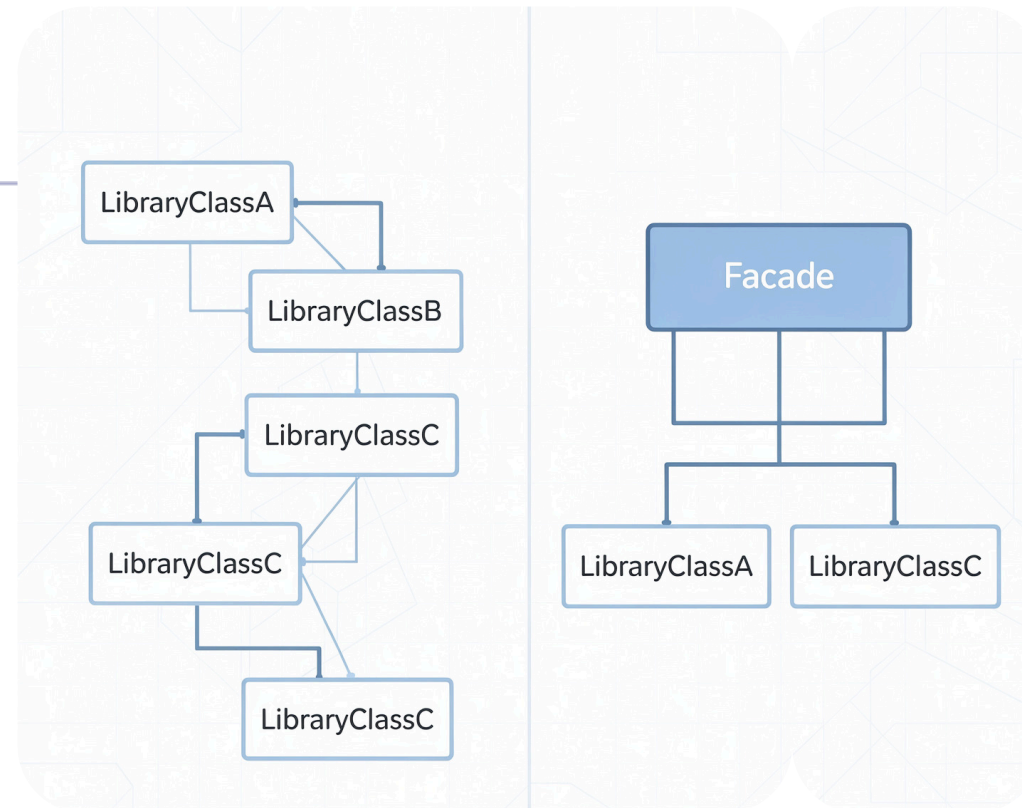
Una aplicación que sube breves vídeos divertidos de gatos a las redes sociales podría utilizar una **biblioteca de conversión de vídeo profesional**.

La Fachada

Lo único que necesita en realidad es una clase con el método simple `codificar(nombreDelArchivo, formato)`. Una vez que crees dicha clase y la conectes con la biblioteca de conversión de vídeo, tendrás tu primera fachada.

Antes

La app llama a múltiples clases de la biblioteca compleja



Después

La app usa una sola clase Fachada que oculta la complejidad

La fachada encapsula la complejidad y expone solo lo esencial.



🌐 CAPÍTULO 4

Analogía en el Mundo Real

Haciendo pedidos por teléfono

Cuando llamas a una tienda para hacer un pedido por teléfono, un **operador** es tu fachada a todos los servicios y departamentos de la tienda. El operador te proporciona una sencilla **interfaz de voz** al sistema de pedidos, pasarelas de pago y varios servicios de entrega.

La Analogía en Detalle



El Cliente (Tú)

Solo necesitas decir qué quieres comprar. No te preocupas por los procesos internos de la tienda.



La Fachada (Operador)

Recibe tu solicitud y coordina internamente con inventario, pagos y logística.

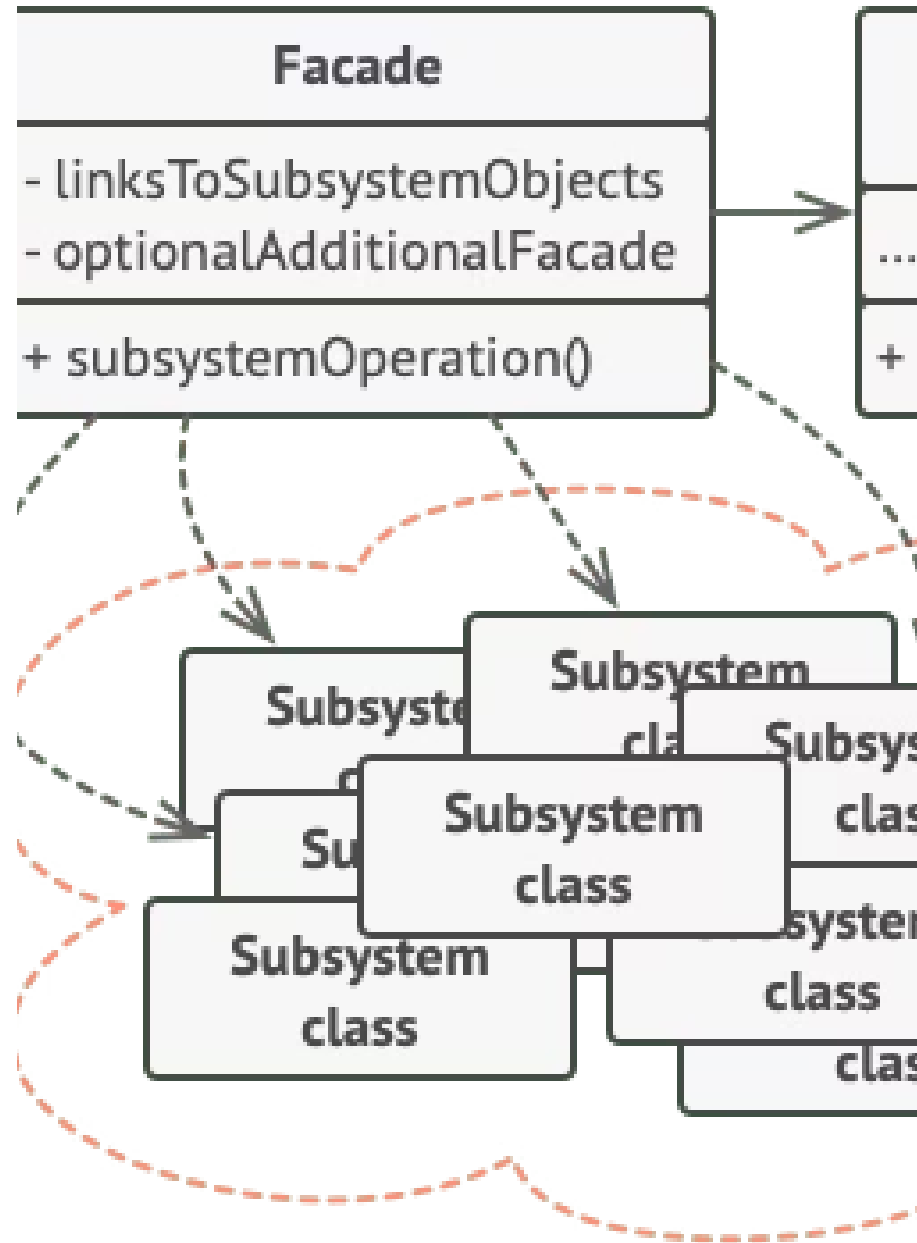


El Subsistema (Tienda)

Múltiples departamentos trabajan juntos, pero tú nunca interactúas directamente con ellos.

Estructura del Patrón

El diagrama muestra los componentes principales del patrón Facade y cómo se relacionan entre sí.



Componentes de la Estructura

1

Facade

Proporciona un práctico acceso a una parte específica de la funcionalidad del subsistema. Sabe a dónde dirigir la petición del cliente y cómo operar todas las partes móviles.

2

Fachada Adicional

Puede crearse para evitar contaminar una única fachada con funciones no relacionadas que podrían convertirla en otra estructura compleja. Pueden utilizarse por clientes y por otras fachadas.

3

Subsistema Complejo

Consiste en decenas de objetos diversos. Para lograr que hagan algo significativo, debes profundizar en los detalles de implementación: inicializar objetos en el orden correcto y suministrarles datos en el formato adecuado.

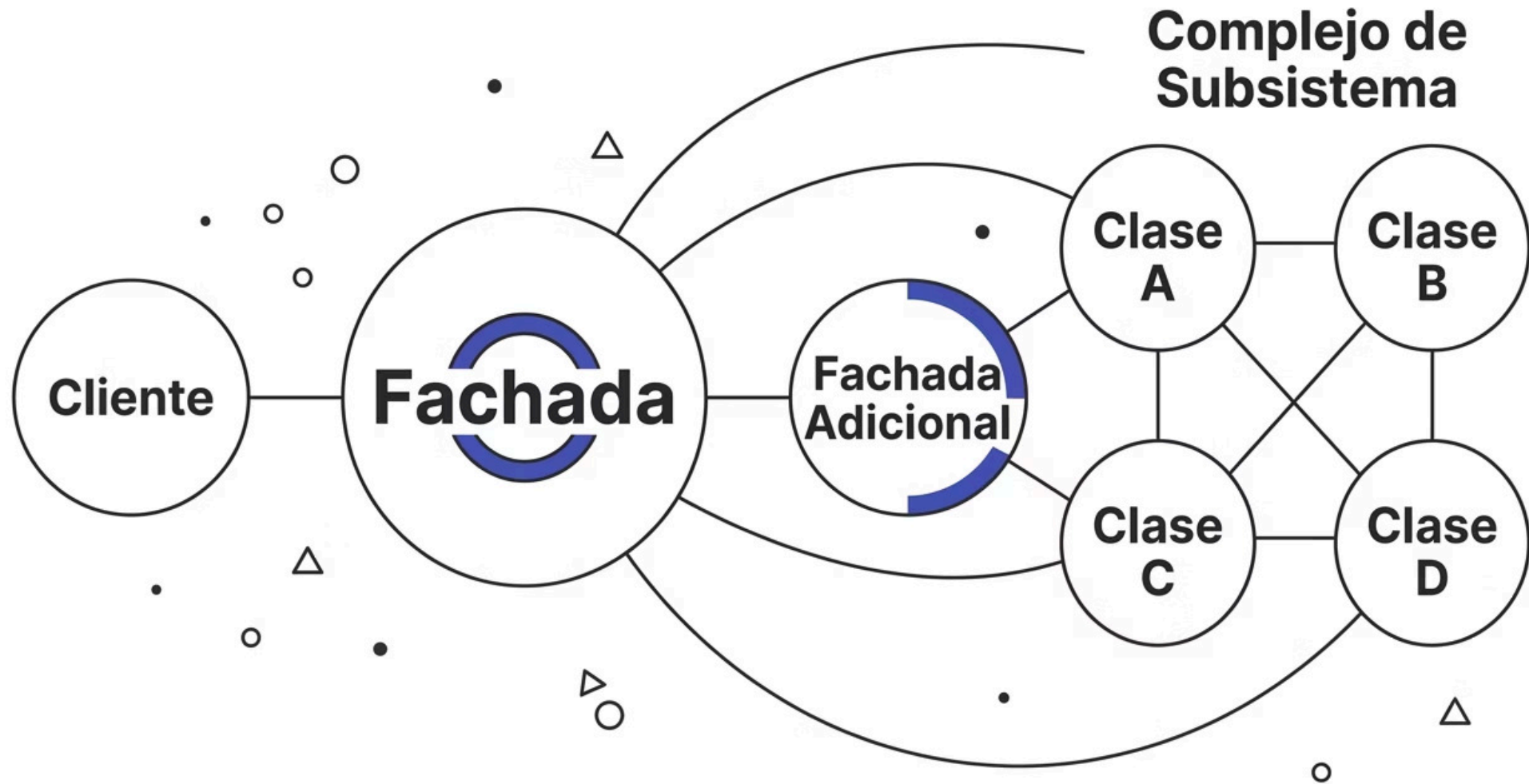
4

Cliente

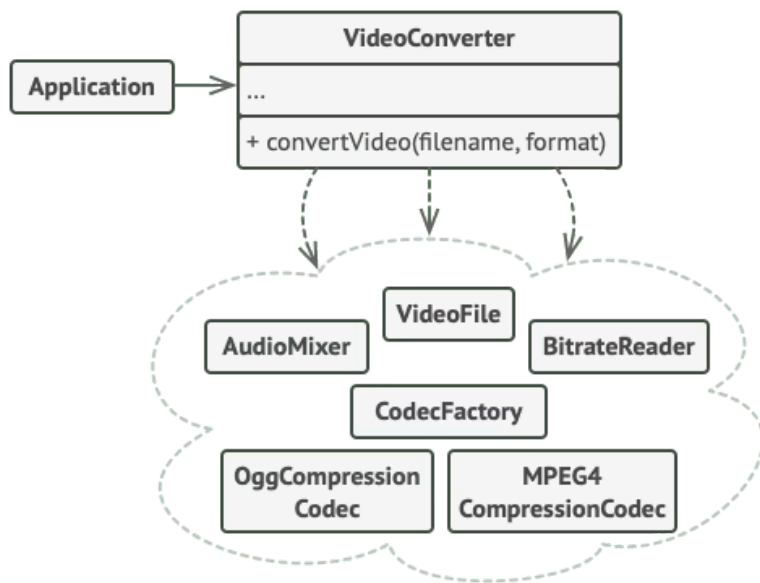
Utiliza la fachada en lugar de invocar directamente los objetos del subsistema.

Detalle Clave de la Estructura

Las clases del subsistema **no conocen la existencia de la fachada**. Operan dentro del sistema y trabajan entre sí directamente.



Pseudocódigo: Contexto



En este ejemplo, el patrón Facade simplifica la interacción con un **framework complejo de conversión de vídeo**.

En lugar de hacer que tu código trabaje con decenas de las clases del framework directamente, creas una clase fachada que **encapsula esa funcionalidad** y la esconde del resto del código.

- ❏ Esta estructura también te ayuda a minimizar el esfuerzo de actualizar a futuras versiones del framework o de sustituirlo por otro. Lo único que tendrías que cambiar es la implementación de los métodos de la fachada.

Pseudocódigo: Clases del Framework

Estas son algunas de las clases de un framework de conversión de vídeo de un tercero. No controlamos ese código, por lo que no podemos simplificarlo.

```
// Estas son algunas de las clases de un framework de conversión
// de video de un tercero. No controlamos ese código, por lo que
// no podemos simplificarlo.
```

```
class VideoFile
// ...
```

```
class OggCompressionCodec
// ...
```

```
class MPEG4CompressionCodec
// ...
```

```
class CodecFactory
// ...
```

```
class BitrateReader
// ...
```

```
class AudioMixer
// ...
```


Pseudocódigo: La Clase Fachada

Creamos una clase fachada para esconder la complejidad del framework tras una interfaz simple. Es una solución de equilibrio entre funcionalidad y simplicidad.

```
// Creamos una clase fachada para esconder la complejidad del  
// framework tras una interfaz simple. Es una solución de  
// equilibrio entre funcionalidad y simplicidad.
```

```
class VideoConverter is  
  method convert(filename, format):File is  
    file = new VideoFile(filename)  
    sourceCodec = (new CodecFactory).extract(file)  
    if (format == "mp4")  
      destinationCodec = new MPEG4CompressionCodec()  
    else  
      destinationCodec = new OggCompressionCodec()  
  
    buffer = BitrateReader.read(filename, sourceCodec)  
    result = BitrateReader.convert(buffer, destinationCodec)  
    result = (new AudioMixer()).fix(result)  
    return new File(result)
```

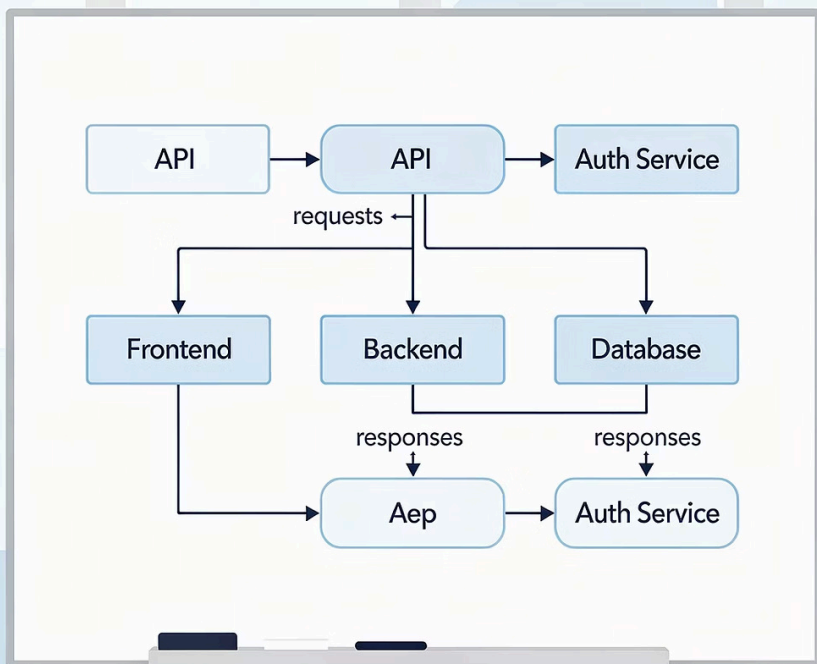
Pseudocódigo: El Cliente

Las clases `Application` no dependen de un millón de clases proporcionadas por el complejo framework. Si decides cambiar los frameworks, sólo tendrás que volver a escribir la clase fachada.

```
// Las clases Application no dependen de un millón de clases
// proporcionadas por el complejo framework. Además, si decides
// cambiar los frameworks, sólo tendrás de volver a escribir la
// clase fachada.
```

```
class Application is
  method main() is
    convertor = new VideoConverter()
    mp4 = convertor.convert("funny-cats-video.ogg", "mp4")
    mp4.save()
```

- ❏ Observa cómo el cliente solo interactúa con `VideoConverter` — una única clase con un método simple — en lugar de manejar `VideoFile`, `CodecFactory`, `BitrateReader` y `AudioMixer` directamente.



👤 CAPÍTULO 7

Aplicabilidad

¿Cuándo deberías utilizar el patrón Facade? Existen dos escenarios principales donde este patrón brilla con especial utilidad.

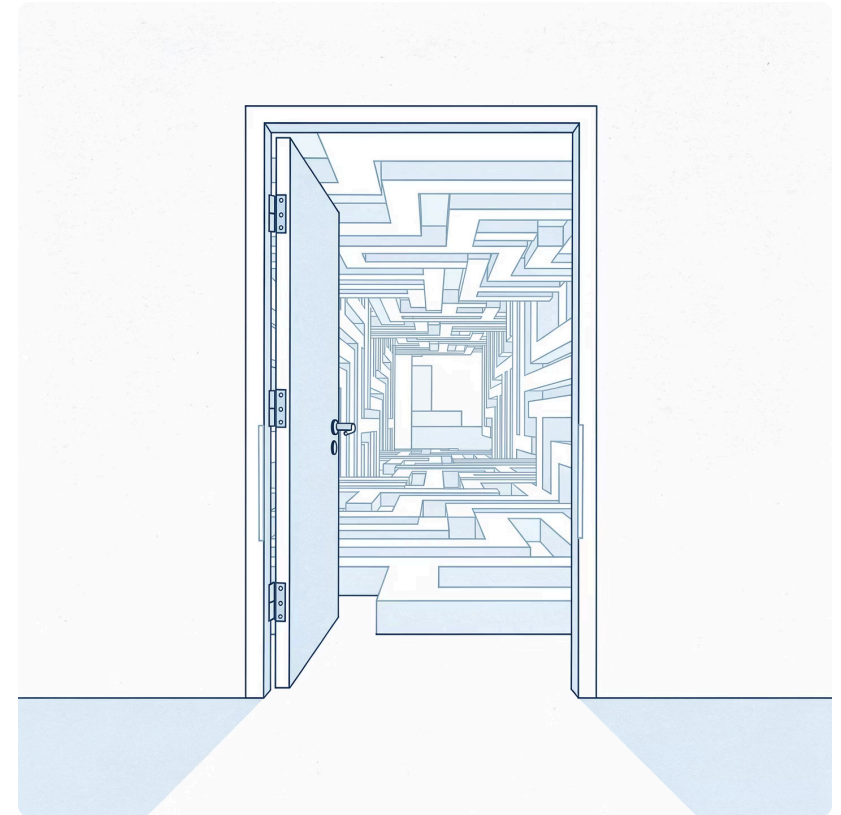
Escenario 1: Interfaz Simplificada

Cuando necesites una interfaz limitada pero directa a un subsistema complejo

A menudo los subsistemas se vuelven más complejos con el tiempo. Incluso la aplicación de patrones de diseño suele conducir a la creación de un **mayor número de clases**.

Un subsistema puede hacerse más flexible y más fácil de reutilizar en varios contextos, pero la cantidad de **código de configuración** que exige de un cliente crece aún más.

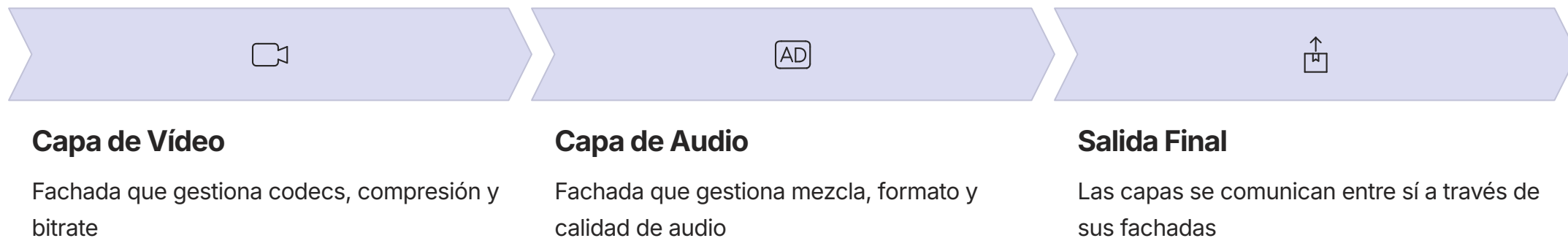
- ❏ El patrón Facade intenta solucionar este problema proporcionando un atajo a las funciones más utilizadas del subsistema que mejor encajan con los requisitos del cliente.



Escenario 2: Estructurar en Capas

Utiliza el patrón Facade cuando quieras estructurar un subsistema en capas.

Crea fachadas para definir puntos de entrada a cada nivel de un subsistema. Puedes reducir el acoplamiento entre varios subsistemas exigiéndoles que se comuniquen **únicamente mediante fachadas**.



Este procedimiento es bastante similar al patrón **Mediator**.

Cómo Implementarlo

1 Verificar viabilidad

Comprueba si es posible proporcionar una interfaz más simple que la que está proporcionando un subsistema existente. Estás bien encaminado si esta interfaz hace que el código cliente sea independiente de muchas de las clases del subsistema.

3 Redirigir el código cliente

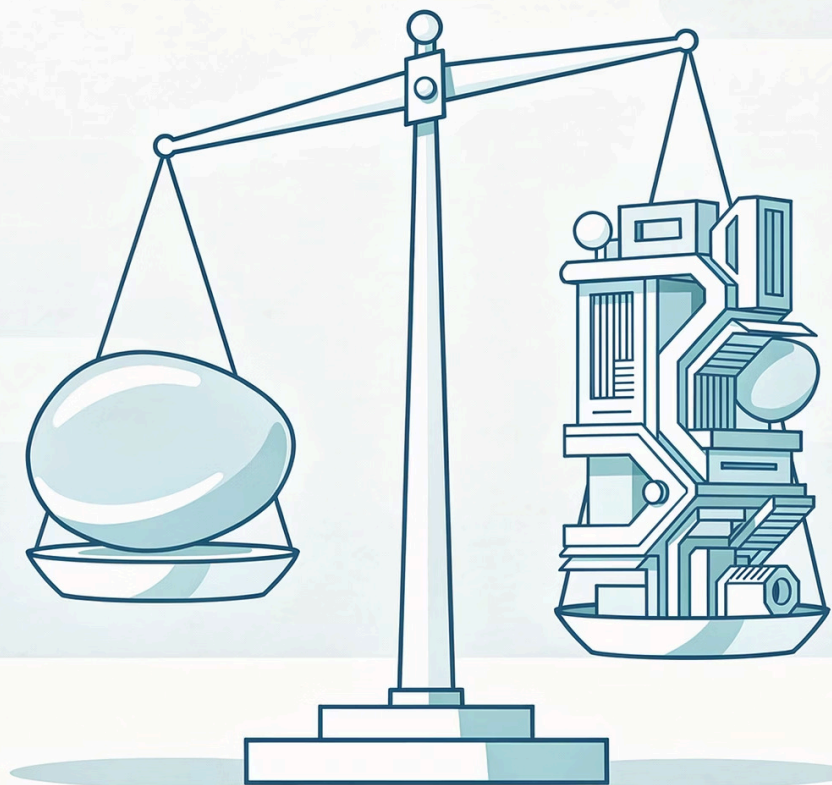
Haz que todo el código cliente se comunique con el subsistema únicamente a través de la fachada. Ahora el código cliente está protegido de cualquier cambio en el código del subsistema.

2 Declarar e implementar la fachada

Declara e implementa esta interfaz en una nueva clase fachada. La fachada deberá redireccionar las llamadas desde el código cliente a los objetos adecuados del subsistema. Será responsable de inicializar el subsistema y gestionar su ciclo de vida.

4 Refactorizar si es necesario

Si la fachada se vuelve demasiado grande, piensa en extraer parte de su comportamiento y colocarlo dentro de una nueva clase fachada refinada.



CAPÍTULO 9

Pros y Contras

Ventajas vs. Desventajas

Ventaja

Puedes **aislar tu código** de la complejidad de un subsistema. El código cliente queda protegido de cambios internos y es más fácil de comprender y mantener.

Desventaja

Una fachada puede convertirse en un **objeto todopoderoso** acoplado a todas las clases de una aplicación, concentrando demasiada responsabilidad en un solo punto.

Relaciones con Otros Patrones

Facade vs. Adapter

Facade define una nueva interfaz para objetos existentes, mientras que **Adapter** intenta hacer que la interfaz existente sea utilizable. Adapter sólo envuelve un objeto; Facade trabaja con todo un subsistema.

Facade vs. Abstract Factory

Abstract Factory puede servir como alternativa a Facade cuando tan solo deseas esconder la forma en que se crean los objetos del subsistema a partir del código cliente.

Facade vs. Flyweight

Flyweight muestra cómo crear muchos pequeños objetos, mientras que **Facade** muestra cómo crear un único objeto que represente un subsistema completo.

Más Relaciones con Otros Patrones

Facade vs. Mediator

Ambos organizan la colaboración entre clases acopladas. **Facade** define una interfaz simplificada sin nueva funcionalidad; el subsistema no conoce la fachada. **Mediator** centraliza la comunicación; los componentes conocen únicamente al mediador y no se comunican directamente.

Facade y Singleton

Una clase fachada a menudo puede transformarse en un **Singleton**, ya que un único objeto fachada es suficiente en la mayoría de los casos.

Facade vs. Proxy

Facade es similar a **Proxy** en que ambos pueden almacenar temporalmente una entidad compleja e inicializarla por su cuenta. Pero Proxy tiene la misma interfaz que su objeto de servicio, lo que los hace intercambiables.

Resumen y Conclusiones

El patrón **Facade** es una herramienta fundamental para gestionar la complejidad en el desarrollo de software.

Simplifica

Proporciona una interfaz sencilla a subsistemas complejos

Desacopla

Aísla el código cliente de los detalles de implementación

Facilita el cambio

Permite actualizar o reemplazar subsistemas con mínimo impacto

Recuerda: una buena fachada no elimina la complejidad, sino que la **organiza** detrás de una interfaz clara y accesible.

